AI Assisted Coding Assignment-20.2

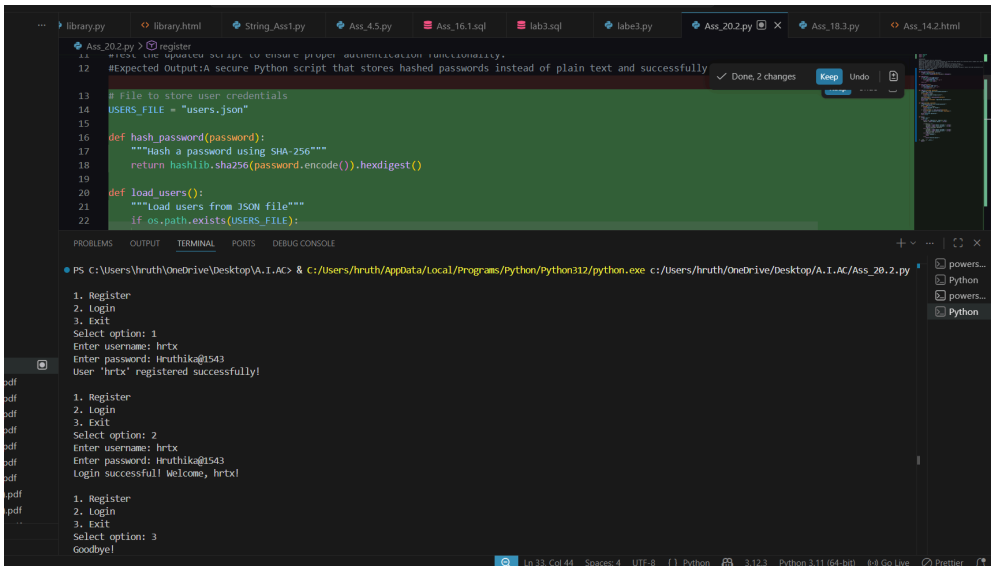
> 2303A51543 BT:29 VELDI.HRUTHIKA 10-4-26

💡 Task 1 – Password Security Enhancement
Task:
Analyze an AI-generated Python program that stores passwords in plain text and improve its security.
Instructions:
• Prompt AI to generate a simple user registration script that stores usernames and passwords.
• Check whether passwords are stored as plain text.
• Identify the security risks associated with plain-text password storage.
• Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
• Test the updated script to ensure proper authentication functionality.
Expected Output:
A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

```
import hashlib
import json
import os

#2303a51543
#Task 1 – Password Security Enhancement
#Generate Python program that stores passwords in plain text and improve its security wit
h a simple user registration script thatstores usernames and passwords.
#Check whether passwords are stored as plain text.
#Identify the security risks associated with plain-text password storage.
#Refactor the program to use secure password hashing (e.g.,hashlib or bcrypt).
#Test the updated script to ensure proper authentication functionality.
#Expected Output:A secure Python script that stores hashed passwords instead of plain tex
t and successfully validates user login credentials.
# File to store user credentials
USERS_FILE = "users.json"

def hash_password(password):
    """Hash a password using SHA-256"""
    return hashlib.sha256(password.encode()).hexdigest()

def load_users():
    """Load users from JSON file"""
    if os.path.exists(USERS_FILE):
        with open(USERS_FILE, 'r') as f:
            return json.load(f)
```

```
    return {}

def save_users(users):
    """Save users to JSON file"""
    with open(USERS_FILE, 'w') as f:
        json.dump(users, f)

def register(username, password):
    """Register a new user with hashed password"""
    users = load_users()
    if username in users:
        print("Username already exists!")
        return False
    users[username] = hash_password(password)
    save_users(users)
    print(f"User '{username}' registered successfully!")
    return True

def login(username, password):
    """Authenticate user with hashed password"""
    users = load_users()
    if username not in users:
        print("Username not found!")
        return False
    if users[username] == hash_password(password):
        print(f"Login successful! Welcome, {username}!")
        return True
    print("Incorrect password!")
    return False

def main():
    """Main menu"""
    while True:
        print("\n1. Register\n2. Login\n3. Exit")
        choice = input("Select option: ").strip()

        if choice == '1':
            username = input("Enter username: ").strip()
            password = input("Enter password: ").strip()
            register(username, password)
        elif choice == '2':
            username = input("Enter username: ").strip()
            password = input("Enter password: ").strip()
            login(username, password)
        elif choice == '3':
            print("Goodbye!")
            break
        else:
            print("Invalid option!")
```

```
if __name__ == "__main__":
    main()
```



Task 2 – Secure API Key Management
Task:
Evaluate an AI-generated script that uses hardcoded API keys and improve its security.
Instructions:
• Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
• Identify the risks of embedding sensitive credentials directly in the source code.
• Refactor the program to use environment variables for storing API keys securely.
• Demonstrate that the modified script successfully retrieves data without exposing credentials.
Expected Output:
A revised script that securely accesses API keys through environment variables instead of hard coding them.

```
#Task 2 – Secure API Key Management
#Generated script that uses hardcoded API keys and improve its security.
#Generate a Python script that connects to an external API using a hardcoded API key.
#Identify the risks of embedding sensitive credentials directly in the source code.
#Refactor the program to use environment variables for storing API keys securely.
#Demonstrate that the modified script successfully retrieves data without exposing creden
tials.
#Expected Output:A revised script that securely accesses API keys through environment var
iables instead of hard coding them.
import os
import requests
```

```python
def fetch_news():
    # Load API key from environment
    api_key = os.getenv("API_KEY")

    if not api_key:
        print("❌ Error: API_KEY environment variable not set!")
        return

    url = "https://newsapi.org/v2/top-headlines"

    params = {
        "country": "in",
        "apiKey": api_key
    }

    try:
        response = requests.get(url, params=params)

        if response.status_code == 200:
            data = response.json()

            print("\n✅ Top News Headlines:\n")

            articles = data.get("articles", [])

            if not articles:
                print("No news found.")
                return

            for i, article in enumerate(articles[:5], start=1):
                print(f"{i}. {article['title']}")

        else:
            print(f"❌ Failed to fetch news. Status Code: {response.status_code}")
            print(response.text)

    except requests.exceptions.RequestException as e:
        print(f"⚠ Network Error: {e}")

if __name__ == "__main__":
    fetch_news()
```

Ass_20.2.py > ...

```python
#Generate a Python script that connects to an external API using a hardcoded API key.
#Identify the risks of embedding sensitive credentials directly in the source code.
#Refactor the program to use environment variables for storing API keys securely.
#Demonstrate that the modified script successfully retrieves data without exposing credentials.
#Expected Output:A revised script that securely accesses API keys through environment variables instead of hard coding them.
import os
import requests

def fetch_news():
    # Load API key from environment
    api_key = os.getenv("API_KEY")

    if not api_key:
        print("❌ Error: API_KEY environment variable not set!")
        return

    url = "https://newsapi.org/v2/top-headlines"

    params = {
        "country": "in",
        "apiKey": api_key
    }

    try:
        response = requests.get(url, params=params)

        if response.status_code == 200:
            data = response.json()

```

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE

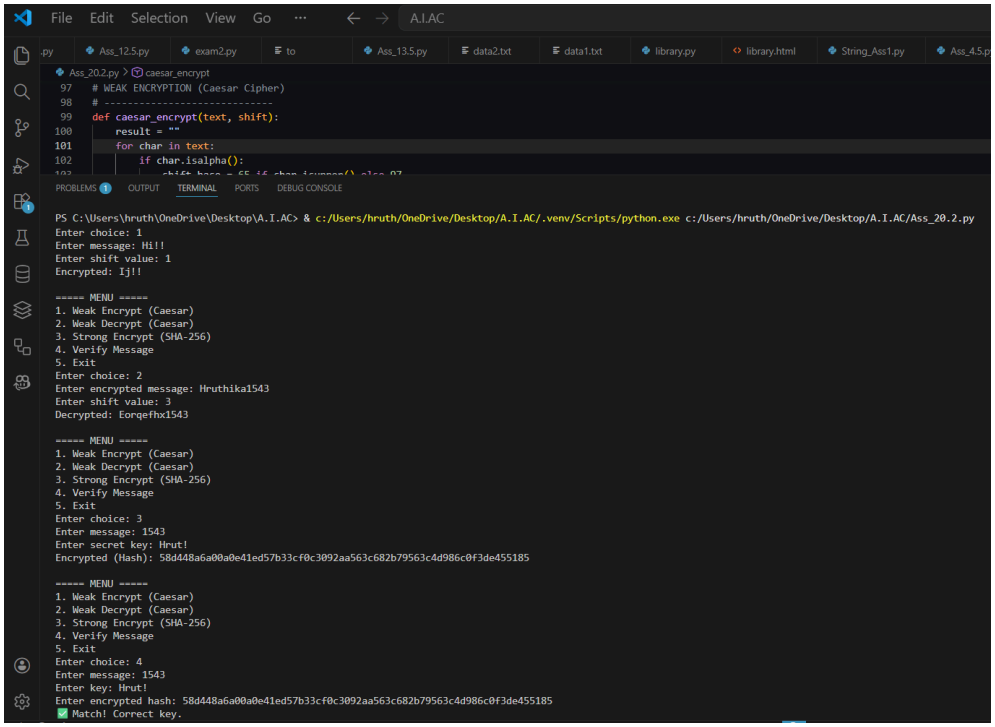
```
✅ Top News Headlines:

No news found.
```

Task 3 – Weak Encryption Detection
Task:
Examine an AI-generated program that uses weak encryption techniques and strengthen it.
Instructions:
• Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
• Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).
• Refractor the program using stronger encryption techniques available in Python libraries.
• Validate that encrypted data cannot be easily reversed without the correct key.
Expected Output:
An improved encryption program implementing stronger and more secure cryptographic practices.

```
#Task 3 – Weak Encryption Detection
#Generate program that uses weak encryption techniques and strengthen it.
#Generate a simple encryption-decryption program using a basic or outdated method.
#Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated al
gorithms).
#Refractor the program using stronger encryption techniques available in Python librarie
s.
#Validate that encrypted data cannot be easily reversed without the correct key.
#Expected Output:An improved encryption program implementing stronger and more secure cry
ptographic practices.
import hashlib
```

```
# ----------------------------
# WEAK ENCRYPTION (Caesar Cipher)
# ----------------------------
def caesar_encrypt(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            shift_base = 65 if char.isupper() else 97
            result += chr((ord(char) - shift_base + shift) % 26 + shift_base)
        else:
            result += char
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)


# ----------------------------
# STRONG "ENCRYPTION" (SHA-256 Hashing)
# ----------------------------
def strong_encrypt(message, key):
    combined = message + key
    encrypted = hashlib.sha256(combined.encode()).hexdigest()
    return encrypted

def verify_message(message, key, encrypted):
    return strong_encrypt(message, key) == encrypted


# ----------------------------
# MAIN PROGRAM
# ----------------------------
def main():
    while True:
        print("\n===== MENU =====")
        print("1. Weak Encrypt (Caesar)")
        print("2. Weak Decrypt (Caesar)")
        print("3. Strong Encrypt (SHA-256)")
        print("4. Verify Message")
        print("5. Exit")

        choice = input("Enter choice: ").strip()

        if choice == '1':
            text = input("Enter message: ")
            shift = int(input("Enter shift value: "))
            print("Encrypted:", caesar_encrypt(text, shift))

        elif choice == '2':
            text = input("Enter encrypted message: ")
```

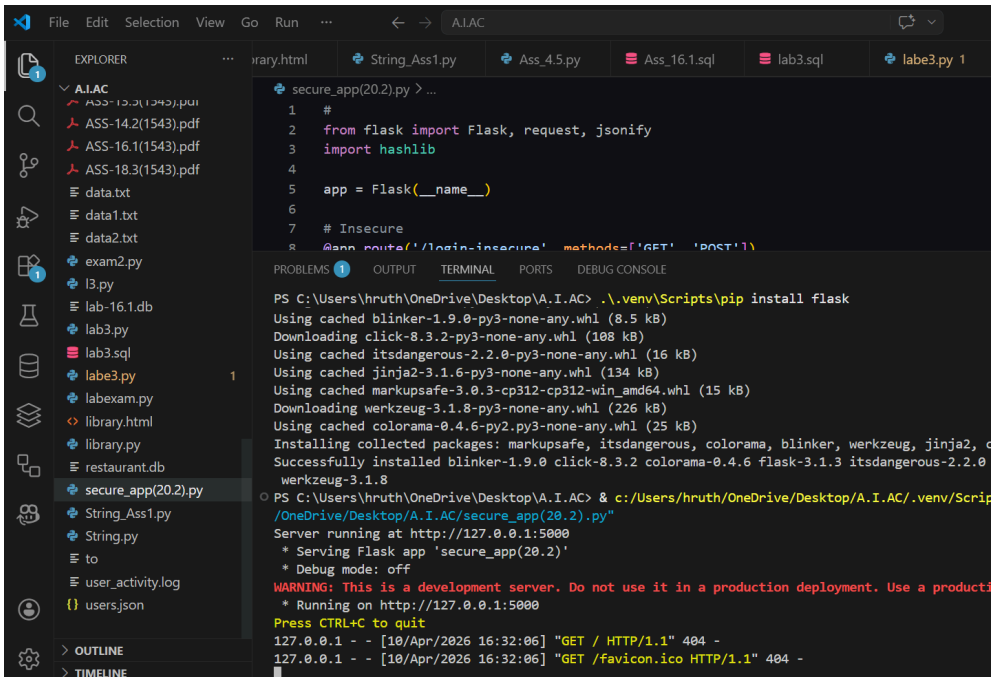
```
            shift = int(input("Enter shift value: "))
            print("Decrypted:", caesar_decrypt(text, shift))

        elif choice == '3':
            msg = input("Enter message: ")
            key = input("Enter secret key: ")
            encrypted = strong_encrypt(msg, key)
            print("Encrypted (Hash):", encrypted)

        elif choice == '4':
            msg = input("Enter message: ")
            key = input("Enter key: ")
            enc = input("Enter encrypted hash: ")
            if verify_message(msg, key, enc):
                print("✅ Match! Correct key.")
            else:
                print("❌ Invalid! Wrong key or message.")

        elif choice == '5':
            print("Exiting...")
            break

        else:
            print("Invalid choice! Try again.")


if __name__ == "__main__":
    main()
```



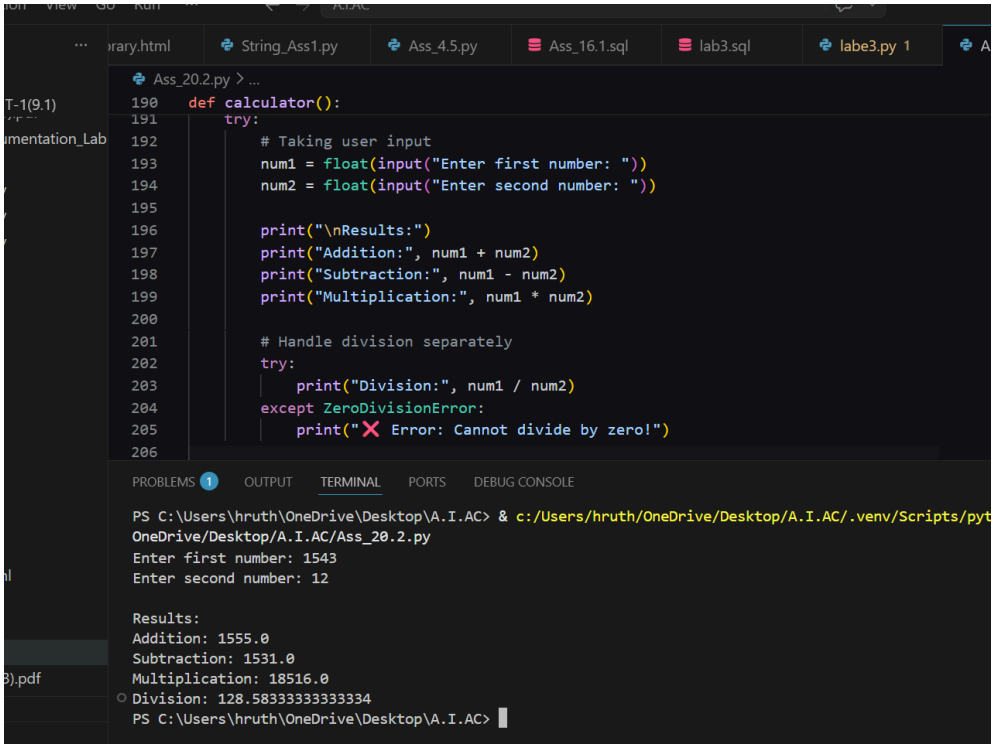
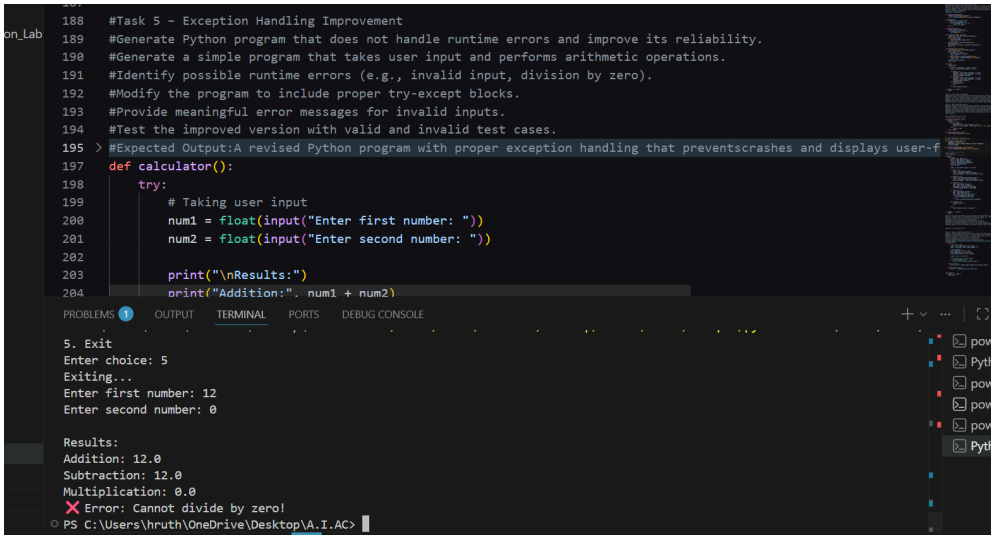
💡 Task 4 – Real-Time Application: Security Review of AI-Generated Web Service
Scenario:
Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).
Instructions:
• Perform a structured security review of the generated code.
• Identify at least three potential vulnerabilities.
• Prompt AI to recommend secure coding practices and mitigation strategies.
• Implement the suggested improvements and test the secured version.
• Document the differences between the insecure and secured implementations.
Expected Output:
A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.

```
#Task 4 – Real-Time Application: Security Review of AI-Generated Web Service
#Scenario:Students select an AI-generated web service snippet (e.g., REST API,file upload
handler, or authentication module).
#Perform a structured security review of the generated code.
#Identify at least three potential vulnerabilities.
#Recommend secure coding practices and mitigation strategies.
#Implement the suggested improvements and test the secured version.
#Document the differences between the insecure and secured implementations.
#Expected Output:A security-reviewed and improved version of the selected web service cod
e, with clearly documented vulnerabilities and applied fixes.

from flask import Flask, request, jsonify
import hashlib

app = Flask(__name__)

# Insecure
@app.route('/login-insecure', methods=['GET', 'POST'])
def login_insecure():
    username = request.values.get('username')
    password = request.values.get('password')

    if username == "admin" and password == "1234":
        return "Login Successful (Insecure)"
    else:
        return "Login Failed (Insecure)"


# Secure
USER_DATA = {
    "admin": hashlib.sha256("1234".encode()).hexdigest()
```

```python
}

@app.route('/login-secure', methods=['GET', 'POST'])
def login_secure():
    username = request.values.get('username')
    password = request.values.get('password')

    if not username or not password:
        return jsonify({"error": "Missing credentials"}), 400

    hashed_input = hashlib.sha256(password.encode()).hexdigest()

    if username in USER_DATA and USER_DATA[username] == hashed_input:
        return jsonify({"message": "Login Successful (Secure)"})
    else:
        return jsonify({"error": "Invalid credentials"}), 401


if __name__ == "__main__":
    print("Server running at http://127.0.0.1:5000")
    app.run(debug=False)
```



💡 Task 5 – Exception Handling Improvement
Task:
Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.
Instructions:
• Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
• Identify possible runtime errors (e.g., invalid input, division by zero).
• Modify the program to include proper try-except blocks.
• Provide meaningful error messages for invalid inputs.
• Test the improved version with valid and invalid test cases.
Expected Output:
A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.

```
#Task 5 – Exception Handling Improvement
#Generate Python program that does not handle runtime errors and improve its reliability.
#Generate a simple program that takes user input and performs arithmetic operations.
#Identify possible runtime errors (e.g., invalid input, division by zero).
#Modify the program to include proper try-except blocks.
#Provide meaningful error messages for invalid inputs.
#Test the improved version with valid and invalid test cases.
#Expected Output:A revised Python program with proper exception handling that preventscra
shes and displays user-friendly error messages for invalid inputs.

def calculator():
    try:
        # Taking user input
        num1 = float(input("Enter first number: "))
        num2 = float(input("Enter second number: "))

        print("\nResults:")
        print("Addition:", num1 + num2)
        print("Subtraction:", num1 - num2)
        print("Multiplication:", num1 * num2)

        # Handle division separately
        try:
            print("Division:", num1 / num2)
        except ZeroDivisionError:
            print("❌ Error: Cannot divide by zero!")

    except ValueError:
        print("❌ Error: Invalid input! Please enter numeric values.")

    except Exception as e:
        print(f"⚠ Unexpected error occurred: {e}")
```

```
# Run program
if __name__ == "__main__":
    calculator()
```